



**POLYTECHNIQUE
MONTRÉAL**

**INF8770 - Technologies multimédia
Hiver 2024**

TP 1

**2045362 – Alexandre Marmen
2060886 – Julien Roux**

Question 1)

Le codage LZW est un choix intéressant d'encodage lorsque l'on souhaite encoder des messages qui contiennent des chaînes répétitives. Si on regarde les données à encoder, certains fichiers ont un bon taux de compression après encodage, notamment les 3 premiers fichiers de textes qui ont des symboles ou sous-chaînes qui se répètent beaucoup et les images 3 et 5 qui ont peut de couleur donc les valeurs de couleur se répèteront beaucoup.

Note: Pour l'image 2 qui est entièrement noir sa taille original est donné comme 0 car pour LZW on utilise le $\log_2$ du nombre de symbole différent or pour une image noir on a qu'un seul symbole (0) donc $\log_2(1) = 0$.

Cependant pour les textes 4 et 5 le taux de compression est faible car le message est composé de beaucoup de caractères différents et avec peu de chaîne qui se répètent. Même chose pour les images 1 et 4 qui ont beaucoup de couleur différentes et avec peu voir pas (image 1) de zone de l'image avec les mêmes couleurs. Pour ces cas là LZW ne donne pas un bon taux de compression et est aussi peu efficace pour les calculs (exécution très longue).

Question 2)

	Temps (secondes)	Taux de compression
Texte 1	0.003	0.119
Texte 2	0.003	0.266
Texte 3	0.003	0.108
Texte 4	0.011	0.001
Texte 5	0.132	0.266
Image 1	480.145	-0.332
Image 2	49.851	NAN (Taille original = 0, raison plus haut)
Image 3	2649	0.560
Image 4	4825	0.151
Image 5	202	0.986

Oui, comme prévu lors de la question 1, pour certains messages l'algorithme proposé par Alice donne de bon résultats de compression mais pour d'autres ces résultats sont très mauvais.

Question 3)

La méthode de compression que nous avons choisie consiste à effectuer l'algorithme LZ77 sur les messages puis Huffman sur les triplets de LZ77. Avec LZ77 on réduit au maximum les sous chaîne qui se répètent puis grâce à hauffman on réduit au maximum le poids lié au triplet identique. Ce choix donne un taux de compression moins bon que LZW pour les messages avec beaucoup de sous chaînes qui se répètent, cependant pour les messages qui ont un taux de compression parfois négatif (message encodé plus grand que celui de base) avec LZW notre méthode permet d'avoir une compression toujours positive peu importe les messages. Notre méthode permet donc d'obtenir une compression moyenne sur tous les messages meilleurs que LZW.

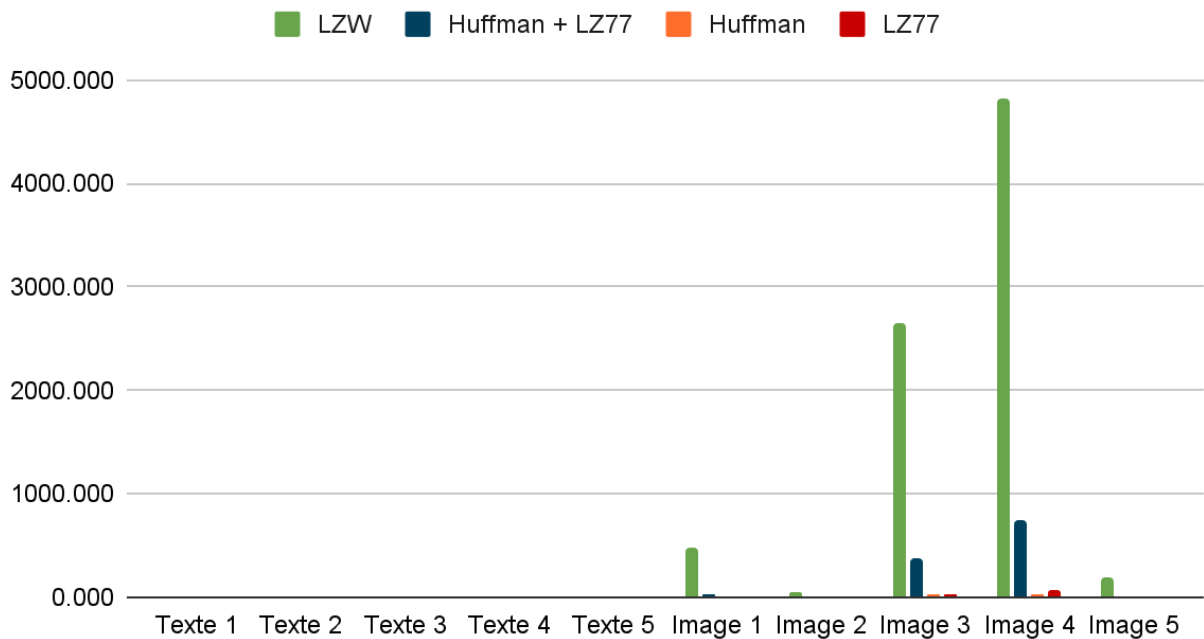
	Temps (secondes)	Taux de compression
Texte 1	0.006	0.194
Texte 2	0.004	0.167
Texte 3	0.007	0.297
Texte 4	0.050	0.359
Texte 5	0.202	0.454
Image 1	36.016	0.395
Image 2	0.123	0.333
Image 3	376.827	0.535
Image 4	738.517	0.515
Image 5	2.222	0.573

On voit aussi que le temps d'exécution est bien meilleur.

Question 4)

En comparant les résultats des taux de compression et temps d'exécution de l'encodage LZW (Alice) et un encodage LZ77 suivi par un encodage de Huffman (notre méthode choisie), on peut voir que LZ77+Huffman a des temps d'exécution beaucoup plus performants que l'encodage LZW pour les images. Cependant, la méthode d'Alice est plus rapide pour les textes. Ceci est probablement dû au fait que les textes sont des fichiers relativement courts où un encodage suivi de l'autre devient contre productif.

Temps d'exécution en secondes



Étant donné l'énorme différence entre les données des temps d'exécution, il serait pertinent de regarder au tableau associé au graphique, les valeurs sont bien sûr en secondes:

	LZW	Huffman + LZ7	Huffman	LZ77
Texte 1	0.002	0.006	0.001	0.001
Texte 2	0.002	0.004	0.001	0.001
Texte 3	0.002	0.007	0.002	0.002
Texte 4	0.010	0.050	0.004	0.031
Texte 5	0.083	0.202	0.018	0.128
Image 1	480.145	36.016	9.394	9.926
Image 2	49.851	0.123	0.137	0.120
Image 3	2649.873	376.827	36.308	31.761
Image 4	4825.681	738.517	32.183	70.165
Image 5	202.367	2.222	4.250	0.855

Pour ce qui en est des taux de compression, la différence entre LZW et LZ77+Huffman est moins importante mais l'encodage LZW est plus efficace seulement pour le texte 2, l'image 3 et l'image 5. Pour les autres messages, l'encodage LZ77+Huffman a des meilleurs taux de compression. Mais surtout LZ77+Huffman produit toujours un fichier plus petit que l'original.

Taux de compression



Références:

[1] "Image processing with Python, NumPy | note.nkmk.me," *note.nkmk.me*.

<https://note.nkmk.me/en/python-numpy-image-processing/>

[2] gabilodeau, "INF8770/Codage LZ77.ipynb at master · gabilodeau/INF8770," *GitHub*, 2018.

<https://github.com/gabilodeau/INF8770/blob/master/Codage%20LZ77.ipynb> (accessed Jan. 27, 2024).

[3] gabilodeau, "INF8770/Codage LZW.ipynb at master · gabilodeau/INF8770," *GitHub*, 2018.

<https://github.com/gabilodeau/INF8770/blob/master/Codage%20LZW.ipynb> (accessed Jan. 27, 2024).

[4] gabilodeau, "INF8770/Codage Huffman.ipynb at master · gabilodeau/INF8770," *GitHub*, 2018.

<https://github.com/gabilodeau/INF8770/blob/master/Codage%20Huffman.ipynb> (accessed Jan. 27, 2024).